

LAPORAN TUGAS PRAKTIKUM STRUKTUR DATA

Disusun oleh

Nama : Muhammad Yusuf Insy

NIM : 2511532003

Dosen Pengampu : DR. Wahyudi,S.T,M.T.

Asisten Praktikum : jovan



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS TAHUN 2026

1. JUDUL PRAKTIKUM

Implementasi Binary Tree dan Graph Traversal (DFS dan BFS) menggunakan Java.

2. TUJUAN PRAKTIKUM

1. Memahami konsep struktur data tree dan graph.
 2. Mengimplementasikan binary tree menggunakan Java.
 3. Mempelajari traversal pada tree (Preorder, Inorder, Postorder).
 4. Mengimplementasikan graph menggunakan adjacency list.
 5. Mempelajari algoritma DFS (Depth First Search) dan BFS (Breadth First Search).
-

3. DASAR TEORI

3.1 Binary Tree

Binary Tree adalah struktur data non-linear yang setiap node memiliki maksimal dua anak, yaitu left child dan right child.

Traversal pada binary tree:

- Preorder : Root $\rightarrow$ Left $\rightarrow$ Right
 - Inorder : Left $\rightarrow$ Root $\rightarrow$ Right
 - Postorder: Left $\rightarrow$ Right $\rightarrow$ Root
-

3.2 Graph

Graph adalah struktur data yang terdiri dari node (vertex) dan edge (hubungan antar node).

Graph dapat bersifat:

- Berarah (directed)
- Tidak berarah (undirected)

Pada praktikum ini digunakan graph tidak berarah dengan representasi adjacency list.

3.3 DFS (Depth First Search)

DFS adalah metode penelusuran graph yang menjelajahi node sedalam mungkin sebelum kembali ke cabang lain. DFS menggunakan pendekatan rekursif.

3.4 BFS (Breadth First Search)

BFS adalah metode penelusuran graph yang menjelajahi node berdasarkan level menggunakan struktur data queue.

4. ALGORITMA / FLOWCHART

4.1 Binary Tree Traversal

- Mulai
 - Cek node
 - Traversal sesuai metode:
 - Preorder
 - Inorder
 - Postorder
 - Rekursif kiri dan kanan
 - Selesai
-

4.2 DFS

- Mulai dari node awal
 - Tandai node sebagai visited
 - Kunjungi tetangga secara rekursif
 - Ulangi hingga semua node dikunjungi
-

4.3 BFS

- Masukkan node awal ke queue
- Ambil node dari queue
- Kunjungi tetangga yang belum dikunjungi

- Masukkan ke queue
 - Ulangi sampai queue kosong
-

5. SOURCE CODE PROGRAM

5.1 Node Binary Tree

Menggunakan class Node_2511532003:

- data
 - left child
 - right child
 - metode traversal (preorder, inorder, postorder)
-

5.2 Binary Tree

Class BTree_2511532003:

- root node
 - search node (DFS tree search)
 - traversal tree
 - count nodes
 - setter & getter root
-

5.3 Tree Driver

Class BtreeDriver_2511532003:

- membuat struktur tree manual
- menambahkan node 1–9
- menghubungkan node secara hirarki
- menampilkan traversal:
 - InOrder
 - PreOrder

- PostOrder
- menampilkan jumlah node

Struktur tree:

```

1
/\
2 3
/\ /\
4 5 6 7
\ /
8 9

```

5.4 Graph Traversal

Class GraphTraversal_2511532003:

- adjacency list menggunakan HashMap
- addEdge (graph tidak berarah)
- DFS rekursif
- BFS menggunakan queue
- print graph

Contoh graph:

- A-B
- A-C
- B-D
- B-E

6. ANALISA PROGRAM

6.1 Binary Tree

Binary Tree yang dibuat menggunakan pendekatan manual (hardcoded), sehingga struktur pohon ditentukan secara langsung melalui penghubungan node.

Traversal:

- Preorder digunakan untuk membaca root terlebih dahulu.
- Inorder menghasilkan urutan terurut pada tree tertentu.
- Postorder digunakan untuk proses evaluasi dari bawah ke atas.

6.2 DFS (Depth First Search)

DFS menggunakan pendekatan rekursif dengan struktur stack implicit. DFS menjelajahi node secara mendalam sebelum berpindah ke cabang lain.

Kelebihan:

- Hemat memori
- Cocok untuk deep exploration

Kekurangan:

- Bisa terjadi infinite loop jika tidak ada visited set

6.3 BFS (Breadth First Search)

BFS menggunakan queue untuk menelusuri node berdasarkan level.

Kelebihan:

- Menemukan shortest path pada graph tak berbobot
- Sistematis berdasarkan level

Kekurangan:

- Membutuhkan memori lebih besar

6.4 Graph Representation

Graph menggunakan adjacency list yang efisien dalam menyimpan hubungan antar node dibanding adjacency matrix.

7. HASIL RUNNING PROGRAM

7.1 Binary Tree Traversal

InOrder:

4 8 2 5 1 6 9 3 7

PreOrder:

1 2 4 8 5 3 6 9 7

PostOrder:

8 4 5 2 9 6 7 3 1

Jumlah node:

9 node

7.2 Graph Traversal

Graph:

A -> B, C

B -> A, D, E

C -> A

D -> B

E -> B

DFS (dari A):

A B D E C

BFS (dari A):

A B C D E

8. KESIMPULAN

1. Binary Tree merupakan struktur data hierarki dengan dua anak maksimal.
2. Traversal tree (preorder, inorder, postorder) memiliki fungsi dan urutan berbeda.
3. Graph dapat direpresentasikan menggunakan adjacency list yang lebih efisien.
4. DFS menggunakan pendekatan kedalaman (stack/rekursif).
5. BFS menggunakan pendekatan level (queue).
6. Kedua struktur data (tree dan graph) sangat penting dalam pemrosesan data non-linear.

7. Seluruh program berhasil dijalankan dengan hasil sesuai konsep teori.